

SIMATS ENGINEERING

ASSESSMENT TOOL 1: AI MODEL DESIGN EXERCISE

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS

COURSE CODE: MLA01

ASSESSMENT TITLE: SMART LOAN APPROVAL – DECISION TREE USING PROLOG

FRONT PAGE

Student Name: S. Roshni

Register Number: 192525185

Student Signature: s.roshni

Course: Artificial Intelligence and Expert Systems

Course Code: MLA01

Assessment: AI Model Design Exercise

CO4: Develop intelligent software agents using suitable agent architectures and learning techniques.

STUDENT DECLARATION

I, S. Roshni (192525185), certify that the submitted work is my original work and that I have followed the academic integrity guidelines specified for this assessment.

Student Name: S. Roshni

Register Number: 192525185

Signature: s.roshni

Date: _____

1. PROBLEM STATEMENT

A bank wants to automatically determine whether a customer is eligible for a loan. The decision depends on important customer attributes such as income, credit score, employment status, loan amount, and repayment history.

The proposed system uses a Decision Tree Classification approach to predict whether a loan application should be Approved or Rejected.

Inputs

- Income
- Credit Score
- Employment Status
- Loan Amount
- Repayment History

Output

- Loan Approved
- Loan Rejected

AI Task

This is a supervised classification problem where previous loan application examples are used to develop decision rules for new applications.

2. OBJECTIVES

- To develop an AI-based loan approval prediction system.
- To identify important factors affecting loan approval.
- To represent customer loan data using Prolog facts.
- To develop decision rules for loan approval.
- To use Gini Index as the attribute-selection measure.
- To test the model with different customer inputs.
- To provide an understandable decision-making process.

3. DATASET DESCRIPTION

The prototype dataset contains customer financial and employment information.

Feature	Data Type	Description
Income	Numerical	Applicant income
Credit Score	Numerical	Creditworthiness
Employment Status	Categorical	Employed/Self-employed/Unemployed
Loan Amount	Numerical	Requested loan amount
Repayment History	Categorical	Good/Average/Poor
Loan Approval	Categorical	Approved/Rejected

The target variable is Loan Approval.

4. DATA PREPROCESSING AND FEATURE ENGINEERING

Step 1 – Data Cleaning

Duplicate and inconsistent records are removed.

Step 2 – Missing Values

Missing numerical values can be replaced using median values. Missing categorical values can be replaced using the most frequent category.

Step 3 – Encoding

Categorical values are represented as Prolog atoms:

- employed
- self_employed
- unemployed
- good
- average
- poor
- approved
- rejected

Step 4 – Feature Selection

Important features are:

- Credit Score
- Income
- Repayment History
- Loan Amount
- Employment Status

Step 5 – Train-Test Concept

For a complete machine-learning implementation, the data can be divided into:

- 80% Training Data
- 20% Testing Data

The assessment specifically requires preprocessing, feature engineering and data splitting to be discussed.

5. SELECTED AI TECHNIQUE AND JUSTIFICATION

Decision Tree

Decision Tree Classification is selected because the problem requires classification into two categories:

Approved / Rejected

A decision tree repeatedly checks important attributes and follows conditions until it reaches a final decision.

Advantages

- Easy to understand.
- Easy to implement.
- Provides interpretable decision rules.
- Suitable for classification.
- Can handle numerical and categorical attributes.
- Useful for decision-support applications.

Attribute Selection Measure – Gini Index

The Gini Index measures the impurity of a node.

Formula

$$\text{Gini} = 1 - \sum(p_i^2)$$

where p_i is the probability of a sample belonging to class i .

A lower Gini value indicates a purer node.

6. MODEL DESIGN / ARCHITECTURE



Input

- Income
- Credit Score
- Employment Status

- Loan Amount
- Repayment History

Processing

```
Prolog Facts
|
Conditions
|
Decision Rules
|
Inference
```

Output

Approved / Rejected

7. ALGORITHM / PSEUDOCODE

```
BEGIN

1. Define the loan application facts.
2. Define income, credit score, employment,
   loan amount and repayment history.
3. Define approved and rejected classes.
4. Calculate Gini impurity for the selected classes.
5. Select important decision attributes.
6. Create decision rules.
7. Give a new customer as input.
8. Apply the decision rules.
9. Generate loan approval prediction.
10. Display Approved or Rejected.
11. Test the system with sample customers.

END
```

8. IMPLEMENTATION USING PROLOG

Software Required

- SWI-Prolog
- Windows/Linux
- Text editor or VS Code

File Name

smart_loan.pl

Prolog Program

```
% SMART LOAN APPROVAL SYSTEM
% Decision Tree Style Implementation in Prolog

% -----
```

```
% LOAN DATA
% -----

loan(1, 60000, 780, employed, 250000, good, approved).
loan(2, 25000, 580, unemployed, 500000, poor, rejected).
loan(3, 50000, 720, employed, 200000, good, approved).
loan(4, 35000, 650, self_employed, 300000, average, approved).
loan(5, 20000, 550, unemployed, 400000, poor, rejected).
loan(6, 75000, 800, employed, 300000, good, approved).
loan(7, 45000, 680, employed, 350000, average, approved).
loan(8, 30000, 590, self_employed, 450000, poor, rejected).
loan(9, 90000, 820, employed, 400000, good, approved).
loan(10, 28000, 600, unemployed, 250000, poor, rejected).
```

```
% -----
% GINI INDEX
% -----
```

```
gini(Approved, Rejected, Gini) :-
    Total is Approved + Rejected,
    Total > 0,
    PA is Approved / Total,
    PR is Rejected / Total,
    Gini is 1 - (PA * PA + PR * PR).
```

```
% -----
% DECISION TREE RULES
% -----
```

```
loan_decision(Income, Credit, Employment,
              Amount, Repayment, approved) :-
```

```
    Credit >= 700,
    Repayment = good,
    Income >= 40000,
    Amount =< Income * 6.
```

```
loan_decision(Income, Credit, Employment,
              Amount, Repayment, approved) :-
```

```
    Credit >= 650,
    Repayment = good,
    Employment \= unemployed,
    Income >= 35000,
    Amount =< Income * 7.
```

```
loan_decision(_, Credit, _, _, _, rejected) :-
    Credit < 650.
```

```
loan_decision(_, _, _, _, poor, rejected).
```

```
loan_decision(Income, _, _, _, _, rejected) :-
    Income < 30000.
```

```
loan_decision(Income, _, _, Amount, _, rejected) :-
    Amount > Income * 8.
```

```
% -----  
% PREDICTION  
% -----
```

```
predict_loan(Income, Credit, Employment,  
             Amount, Repayment, Result) :-
```

```
    loan_decision(  
        Income,  
        Credit,  
        Employment,  
        Amount,  
        Repayment,  
        Result  
    ).
```

```
% -----  
% SAMPLE TEST  
% -----
```

```
test :-
```

```
    predict_loan(  
        60000,  
        780,  
        employed,  
        250000,  
        good,  
        R1  
    ),
```

```
    write('Customer 1: '),  
    write(R1),  
    nl,
```

```
    predict_loan(  
        25000,  
        580,  
        unemployed,  
        500000,  
        poor,  
        R2  
    ),
```

```
    write('Customer 2: '),  
    write(R2),  
    nl,
```

```
    predict_loan(  
        50000,  
        720,  
        employed,  
        200000,  
        good,  
        R3  
    ),
```

```
    write('Customer 3: '),
```

```

    write(R3),
    nl.

% -----
% GINI TEST
% -----

gini_example :-

    gini(6, 4, G),

    write('Gini Index = '),
    write(G),
    nl.

```

9. HOW TO RUN THE PROLOG PROGRAM

Step 1

Install SWI-Prolog.

Step 2

Open a text editor.

Step 3

Paste the above program.

Step 4

Save the file as:

smart_loan.pl

Step 5

Open SWI-Prolog.

Step 6

Load the program:

```
?- [smart_loan].
```

If successfully loaded:

```
true.
```

Step 7

Run the test:

```
?- test.
```

Expected Output

```

Customer 1: approved
Customer 2: rejected
Customer 3: approved

```


10. SAMPLE PREDICTIONS

Test Case 1

```
?- predict_loan(60000,780,employed,250000,good,R).
```

Output:

```
R = approved.
```

Test Case 2

```
?- predict_loan(25000,580,unemployed,500000,poor,R).
```

Output:

```
R = rejected.
```

Test Case 3

```
?- predict_loan(50000,720,employed,200000,good,R).
```

Output:

```
R = approved.
```

Gini Test

```
?- gini(6,4,G).
```

Output:

```
G = 0.48.
```

11. EXPERIMENTAL RESULTS

Test Case	Income	Credit Score	Employment	Repayment	Prediction
Customer 1	60000	780	Employed	Good	Approved
Customer 2	25000	580	Unemployed	Poor	Rejected
Customer 3	50000	720	Employed	Good	Approved

The experimental section should present suitable test cases and results, as required by the assessment structure.

12. PERFORMANCE EVALUATION

The model can be evaluated using:

Accuracy

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

Precision

$$\text{Precision} = TP / (TP + FP)$$

Recall

$$\text{Recall} = TP / (TP + FN)$$

F1-Score

$$F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

These are among the evaluation metrics specified in the assessment.

Note: Exact numerical accuracy, precision, recall and F1 values should only be reported after testing against a defined labelled test dataset.

13. RESULT ANALYSIS AND INTERPRETATION

The Prolog-based decision system provides an understandable loan decision.

A customer with:

- High credit score
- Good repayment history
- Sufficient income
- Reasonable loan amount

is more likely to be classified as Approved.

A customer with:

- Low credit score
- Poor repayment history
- Low income
- Excessive loan amount

is more likely to be classified as Rejected.

The major advantage is that the decision can be explained using the conditions contained in the Prolog rules.

14. LIMITATIONS

- The prototype uses a small dataset.
- Decision rules are manually defined.
- It does not automatically learn a complete tree from a large dataset.
- Real financial data may contain bias.
- Customer financial conditions can change.
- Real banking systems require strong privacy and security.
- The prediction should not be the only factor used for a real loan decision.

15. FUTURE ENHANCEMENTS

- Use a larger real-world dataset.
- Automatically generate decision-tree rules.
- Connect Prolog with a database.
- Add cross-validation.

- Compare Decision Tree with Random Forest.
- Develop a web-based interface.
- Add explainable AI.
- Add fairness and bias detection.
- Add fraud detection.
- Deploy the system in a secure banking environment.

16. CONCLUSION

The Smart Loan Approval System demonstrates how a Decision Tree-style AI model can be implemented using Prolog. The system represents loan applications as facts and uses logical decision rules to classify customers as Approved or Rejected.

The model considers income, credit score, employment status, loan amount and repayment history. The Gini Index is used as the selected attribute-selection measure.

The Prolog implementation provides a simple, transparent and understandable approach to loan decision-making. The system can be enhanced in the future using larger datasets, automatic decision-tree learning, database connectivity and explainable AI.

17. REFERENCES

- Stuart Russell and Peter Norvig, Artificial Intelligence: A Modern Approach.
- SWI-Prolog Documentation.
- Artificial Intelligence and Expert Systems course materials.
- SIMATS Engineering – Assessment Tool 1: AI Model Design Exercise.

18. GITHUB REPOSITORY

GitHub Repository: <https://github.com/192525185simats-cyber/SLOT-D-MLA0104-AI/tree/main/assessment/co4>

The repository should contain:

- smart_loan.pl
- README file
- Dataset/source information
- Sample queries
- Output screenshots
- Documentation
- Results

The assessment specifically requires the repository to contain the complete source code, dataset information, implementation, results and README.

BACK PAGE – RUBRICS EVALUATION

STUDENT EVALUATION SHEET

S.No.	Evaluation Criterion	Weightage	Marks Obtained
1	Problem Analysis and Domain Understanding	10%	
2	Dataset Selection and Data Understanding	10%	
3	Data Preprocessing and Feature Engineering	10%	
4	AI Technique Selection and Justification	10%	
5	Model Design / Architecture	10%	
6	Algorithm / Pseudocode Development	5%	
7	Model Implementation and GitHub Repository	15%	
8	Experimental Design and Results	10%	
9	Performance Evaluation and Metrics	10%	
10	Result Analysis and Interpretation	5%	
11	Limitations and Future Enhancements	5%	
12	Documentation and Technical Presentation	10%	
	TOTAL	100%	____ / 100

These weightages follow the assessment rubric provided in the uploaded document.